华南农业大学数学与信息学院

程序设计与算法综合实习Ⅱ课程设计

学院	数学与信息学院	**班级**	25 计机 6 班	**学号**	202525310603（组长） 202525310607 202525310606	**姓名**	陈忠颖 李鸿伟 赖霖
实验题目	**数据结构在游戏编程中的应用——俄罗斯方块的设计与实现**						
自我评价	本次课程设计以三人小组的形式完成了一个基于 raylib 的图形版俄罗斯方块程序，三个人按底层内核、框架交互、功能集成三层分工协作；李鸿伟负责棋盘、方块、队列等底层数据结构和碰撞、消行等核心算法，陈忠颖负责图形环境、程序主框架、渲染工具、中文显示和键鼠交互，赖霖负责落点影子、难度选择、计分升级和排行榜文件读写，并完成整体集成测试；开发过程中各自先在约定的接口上并行推进，再把模块逐层拼到一起，期间解决了踢墙卡死、中文乱码、排行榜排序等问题，程序经过多组测试运行稳定，三个人也在分工对接中对数据结构的应用和团队协作有了更具体的认识。						
教师评语							
成绩							

目　录

一、分析题目要求. 1
（一）程序设计任务. 1
（二）小组分工与任务划分. 1
（三）函数说明. 2
（四）输入形式与输入值范围. 3
（五）输出形式. 4
（六）程序所能达到的功能. 4
二、解题思路. 4
（一）总体设计与三层架构. 4
（二）底层数据结构与核心算法（李鸿伟）. 5
（三）主框架与交互实现（陈忠颖）. 7
（四）扩展功能与持久化（赖霖）. 8
（五）模块对接与系统集成. 10
三、调试分析. 10
（一）底层内核问题（李鸿伟）. 10
（二）框架交互问题（陈忠颖）. 10
（三）扩展功能问题（赖霖）. 11
（四）集成测试与测试数据. 11
（五）算法效率分析. 12
（六）改进设想. 12
（七）团队协作体会. 13
四、附录. 13

一、分析题目要求

（一）程序设计任务

本次课程设计选了经典的俄罗斯方块作为实现对象，目标是把课堂上学到的数组、队列、顺序表等数据结构用到一个完整可玩的图形游戏当中，程序由三人小组共同完成，代码用 C++17 编写，界面部分借助 raylib 图形库完成；一局游戏从主菜单开始，玩家选定难度后，方块按固定节奏从棋盘顶部出现并不断下落，玩家通过键盘控制方块的位置和朝向，方块落到底部或堆到已有方块上就固定下来，把横向凑满的行消掉后能得到相应的分数，方块堆到顶部时这一局结束，结束后的成绩会写入本地排行榜。

（二）小组分工与任务划分

表 1　小组分工对照表

成员	主要开发工作	报告负责内容	主要调试问题
李鸿伟 （成员 A）	设计全部底层数据结构，实现方块初始化、碰撞检测、移动旋转踢墙、环形队列、多行消行内核算法	数据结构、碰撞检测、方块队列、算法效率分析	旋转踢墙卡死、多行消行漏行、方块出生越界判定
陈忠颖 （成员 B、组长）	Raylib 环境搭建，程序主框架、渲染工具、中文显示、基础界面与输入交互	输入输出、主循环流程、收集全部运行截图	中文乱码、基础按键交互 bug
赖霖 （成员 C）	开发落点影子、难度选择、排行榜文件 IO、计分升级，集成测试、报告全文整合	函数说明表、扩展功能、测试用例、改进设想	影子渲染异常、难度速度异常、排行榜读写排序、文件容错

从表里能看到三个人的工作按“底层内核、框架交互、功能集成”的顺序衔接，李鸿伟负责数据结构和核心算法，是整个程序的地基；陈忠颖处在中间的框架层，对上承接内核的算法调用，对下给扩展功能提供渲染和交互的基础；赖霖在框架之上完成玩法扩展和数据持久化，最后把三个人的模块集成到一起。

（三）函数说明

表 2　全部函数说明表

函数名	参数	返回值	功能说明	负责人
InitPieces	无	void	初始化七种方块的四种旋转形态	李鸿伟
resetBoard	无	void	把棋盘全部格子置空	李鸿伟
legal	shape, form, bx, by	bool	判断方块在目标位置是否越界或碰撞	李鸿伟
moveBy	dx, dy	bool	试探目标位置，合法才提交坐标	李鸿伟
rotate	无	void	旋转方块并按踢墙偏移尝试	李鸿伟
lockPiece	shape, form, bx, by	void	把活动方块写进棋盘	李鸿伟
fullRows	无	vector<int>	扫描并返回全部满行	李鸿伟
removeRows	rows	void	删除满行并把非满行压实	李鸿伟
qInit / qPop / qFront	无	void / PiecePair	环形队列的初始化、取出和查看	李鸿伟
spawn	无	void	从队列生成方块并做出生判定	李鸿伟
Shade	Color c, float f	Color	按倍数把颜色调亮或调暗	陈忠颖
DrawCell	x, y, sz, c	void	画带高光暗边的立体方块	陈忠颖
DrawGhostCell	x, y, sz, c	void	画落点影子用的半透明方块	陈忠颖
TextCenter	f, text, cx, y, size, c	void	以指定点为中心画文字	陈忠颖
CollectCodePoints	无	vector<int>	解析文案并收集字符码点	陈忠颖
update / draw	dt / 无	void	按状态分发更新和绘制	陈忠颖
updateMenu / chooseMenu	无 / int i	void	主菜单的交互与动作	陈忠颖

函数名	参数	返回值	功能说明	负责人
drawMenu / drawDifficulty / drawRank	无	void	菜单、难度、排行榜界面绘制	陈忠颖
drawPause / drawGameOver Screen	无	void	暂停与结束遮罩绘制	陈忠颖
drawGame / drawPanel / drawNext	无	void	主画面、信息栏和预览绘制	陈忠颖
main	无	int	初始化资源、运行主循环、退出清理	陈忠颖
shadowY	无	int	向下试探影子方块的落点	赖霖
dropInterval	无	float	按难度和等级算下落间隔	赖霖
addLines	int n	void	按消行数加分、升级	赖霖
startGame	int idx	void	重置本局状态并开始新局	赖霖
loadRanks / saveRanks	无	void	排行榜的文件读取和写入	赖霖
addScore	int score	void	合并成绩、降序排序、取前五	赖霖
saveHigh	无	void	把最高分写回文件	赖霖
endGame	无	void	结算成绩并切到结束界面	赖霖
updatePlaying / updateClearing	float dt	void	游戏中和消行动画的更新	赖霖

（四）输入形式与输入值范围

程序的输入分三类，分别是键盘、鼠标和文件读取。

键盘上，左右方向键把方块左右移动，下方向键让方块加速下落，上方向键或空格键把方块顺时针旋转 90 度，上下方向键在菜单里移动选中项，回车键用于确认，P 键或 S 键暂停，R 键重开一局，Esc 键退回上一级菜单；鼠标在菜单和难度界面可以悬停选中、点击确认；文件输入有两个，high_score.txt 保存历史最高分，leaderboard.txt 保存前五名成绩，程序启动时读入，文件缺失或内容

不完整时按 0 处理。

数值范围方面，棋盘固定为 16 列 20 行，七种方块的形状编号取 0 到 6，旋转形态编号取 0 到 3，难度下标取 0 到 2，分数和行数都是非负整数。

（五）输出形式

程序输出一个 800×650 像素的图形窗口，窗口里会出现主菜单、难度选择、排行榜、游戏主画面、暂停遮罩、结束遮罩共六种画面；游戏主画面左侧是 16×20 的棋盘，右侧信息栏依次显示接下来三个方块的预览、当前分数、最高纪录、当前等级、消除行数和按键说明；游戏结束或退出时，最高分和排行榜会写回两个文本文件。

（六）程序所能达到的功能

一是完整的核心玩法，方块能移动、旋转、软降、锁定和消行，规则与经典俄罗斯方块一致；

二是三档难度和随等级变化的下落速度，玩家可以按自己的水平选择；

三是落点影子和三格预览，帮助玩家提前判断位置；

四是计分、等级和最高分记录，成绩能在本地长期保存；

五是排行榜功能，前五名成绩按分数排序并持久化到文件；

六是完整的中文界面，键盘和鼠标都能操作，暂停、重开、返回菜单等流程齐全。

二、解题思路

（一）总体设计与三层架构

程序按“每帧更新、每帧重画”的方式运行，主循环每跑一轮，先按当前状态处理输入和逻辑，再把画面整屏重画一遍；游戏的七种界面用一个枚举表示，分别是主菜单、难度选择、排行榜、游戏中、暂停、消行动画和游戏结束，更新和绘制都按当前状态分发到对应函数，新增界面只需要加一个状态和一对更新、绘制函数。

整个程序在代码组织上分成三层，底层是棋盘、方块和队列等数据结构以及

碰撞、消行等核心算法，中间层是主循环、状态机和渲染工具，上层是落点影子、难度、计分和排行榜等扩展功能；三层之间靠约定好的函数接口对接，数据自下而上提供状态、自上而下发起调用，三个人的工作正好对应这三层，可以并行开发再逐层拼合。

【此处插入截图：图 1　框架渲染的游戏主画面（含棋盘、预览和信息栏）】

图 1　框架渲染的游戏主画面（含棋盘、预览和信息栏）

（二）底层数据结构与核心算法（李鸿伟）

1. 棋盘与方块的数据结构

棋盘用二维数组 cell[20][16] 表示，第一维是行、第二维是列，空格保存 -1，已锁定的格子保存 0 到 6 的形状编号，坐标访问直接、内存连续，适合每帧绘制和碰撞检测；七种方块的全部形态保存于四维数组 g_piece[7][4][4][4]，四个下标依次表示形状、旋转形态、模板行和模板列，值为 1 表示该位置有方块，程序只手工填写每种方块的 0 号形态，再用顺时针旋转公式 new[i][j] = old[3-j][i] 批量生成其余三种形态，减少重复代码并保证旋转规则一致。

2. 统一碰撞检测

合法性检测函数 legal 是移动、旋转、出生和影子计算共用的底层函数，它遍历方块 4×4 模板中值为 1 的格子，把模板坐标换算成棋盘坐标，横向越出左右边界或纵向越过底部就返回 false，与已锁定方块重叠也返回 false；位于棋盘上方的格子（y 小于 0）放行，因为方块刚从顶部出生时本来就有一部分在屏幕外，所有试探操作都先调用 legal，只有成功才提交坐标修改。

```
legal(shape, form, bx, by):
    for 模板中每个有方块的格子 (i, j)
        x = bx + j;  y = by + i
        if x 越左右边界 or y 越底部: return false
        if y < 0: 继续
        if cell[y][x] 已有方块: return false
    return true
```

3. 移动、旋转与踢墙

移动函数 moveBy 先试探目标位置，合法才提交坐标；旋转函数先算出下一形态，再按候选偏移依次尝试原地、向左 1 格、向右 1 格、向左 2 格、向右 2 格，并额外尝试向上抬 1 格，找到第一个合法位置后一次性更新坐标和形

态，所有候选都失败就保持原样，这样方块贴墙时也能尽量旋转成功。

```
rotate:
    nf = (form + 1) mod 4
    offsetX = {0, -1, 1, -2, 2}
    for oy in {0, -1}
        for k = 0 to 4
            if legal(shape, nf, px + offsetX[k], py + oy)
                提交坐标和形态，返回
    全部失败：保持原样
```

4. 方块生成的环形队列

方块供给用容量为 8 的数组实现环形队列，队头指针指向下一个要取出的方块，队尾指针指向下一个可写位置，另设一个计数记录当前元素个数；下标更新统一对容量取模，队尾走到数组末端后能绕回开头，队头取出一个方块后立刻在队尾补一个随机方块，队列长度保持不变，同时支持方块按顺序出现和右侧三格预览。

```
qPop:
    result = queue[head]
    head = (head + 1) mod 容量; count 减 1
    queue[tail] = 随机方块
    tail = (tail + 1) mod 容量; count 加 1
    return result
```

5. 多行消行与压缩

方块锁定后先调用 fullRows 从上到下扫描全部 20 行，把所有满行的行号收集到一个动态数组；消行函数先建立一个行标记数组把满行标出来，再用一个写指针从底部开始，把非满行逐行写到写指针指向的位置，满行直接跳过，最后把写指针上方空出的行全部置空；扫描和压缩分开处理，连续消除多行时也不会因为行号变化而漏行。

```
removeRows(rows):
    标记全部满行
    write = 棋盘高度 - 1
    for y 从底部到顶部
        if 该行是满行: 跳过
        cell[write] = cell[y]; write 减 1
    把 write 以上的行全部置空
```

【此处插入截图：图 2　多行消行闪烁画面】

图 2　多行消行闪烁画面

（三）主框架与交互实现（陈忠颖）

1. 主循环与状态机

程序入口先创建 800×650 的窗口并把帧率固定为 60，再初始化方块表、加载字体、读入最高分和排行榜，之后进入主循环；主循环每轮先取出距上一帧经过的时间，再调用更新函数，随后开始绘制、调用绘制函数、结束绘制，玩家关闭窗口或选择退出时循环结束，退出前保存最高分、释放字体并关闭窗口。

```
main:
    初始化窗口，固定帧率为 60
    初始化方块表，加载字体
    读入最高分和排行榜
    while 窗口未关闭且未选择退出
        dt = 距上一帧经过的时间
        update(dt)
        开始绘制; draw(); 结束绘制
    保存最高分，释放字体，关闭窗口
```

2. 渲染工具

渲染工具是所有界面共用的基础，颜色调整函数把颜色的三个分量按倍数缩放并夹在 0 到 255 之间，用于方块的高光和暗边；立体方块函数先画方块主体，再在顶部和左侧画调亮的窄边、底部和右侧画调暗的窄边，方块看起来有凸起的立体感；居中文字函数先测量文字宽高，再把起点向左偏移半个字宽，文字就能以指定点为中心画下。

3. 中文字体按需加载

界面里要用到的中文文案先集中放进一个文案表，程序启动时从表里收集全部字符的码点，再用扩展字体加载函数只加载这些码点；码点收集先塞入 ASCII 可打印字符和箭头等符号，再逐段扫描文案，按 UTF-8 的编码规则手动解出每个字符的码点，1 字节、2 字节、3 字节、4 字节分别处理，收集结果去重后交给字体加载；字体文件选用 Windows 自带的黑体 ttf，加载失败时退回默认字体，保证程序不会因为字体问题起不来。

```
CollectCodePoints:
    先塞入 ASCII 可打印字符和箭头、标点
    for 文案表中的每段文字
        按字节扫描，判断 UTF-8 首字节类型
        1、2、3、4 字节分别按规则拼接码点
        码点放入集合去重
```

```
    返回排序后的码点列表
```

4. 键盘长按与鼠标点选

键盘输入区分按下瞬间、按住状态和释放瞬间，点按操作只在按下瞬间响应一次，长按操作需要连续触发；左右移动和软降都支持长按，按下时先移动一格并记下按住状态，按住期间用帧时间累计，累计到 0.16 秒后每 0.05 秒重复移动一次，软降的重复间隔为 0.05 秒，按键释放时清掉按住标志，避免松手后方块还在动；鼠标每轮先取出当前位置，再和按钮区域数组逐个做矩形包含判断，落在哪个按钮上就把选中项切到哪个，左键按下时确认，鼠标悬停和键盘移动的选中状态始终保持一致。

```
左右移动按键：
    按下瞬间：移动一格，记下按住，计时清零
    按住期间：计时 += dt
        计时达到 0.16：再移动一格，计时减 0.05
    释放瞬间：清掉按住标志
```

【此处插入截图：图 3　主菜单界面】

图 3　主菜单界面

【此处插入截图：图 4　难度选择界面】

图 4　难度选择界面

【此处插入截图：图 5　暂停界面】

图 5　暂停界面

【此处插入截图：图 6　游戏结束界面】

图 6　游戏结束界面

（四）扩展功能与持久化（赖霖）

1. 落点影子

落点影子只用于提示，不提前修改棋盘，也不移动正在下落的方块；实现时把临时纵坐标设为方块当前的行，再反复调用底层合法性函数试探下一格，下一格仍合法就把临时坐标加一，一旦不合法就停止，临时坐标就是影子的落点；绘制时只对模板中值为 1 的格子画半透明填充和较明显的描边，与真实方块的实体填充区分开。

```
shadowY:
    sy = py
    while legal(shape, form, px, sy + 1)
```

```
        sy 加 1
    return sy
```

2. 难度选择与下落速度

每种难度用一个结构体保存名称、说明、初始下落间隔、每升一级减少的间隔、间隔下限和每升一级所需消行数，三档参数集中放在配置数组里；选定难度后，下落间隔由一个统一公式计算，用初始间隔减去等级带来的缩短量，再用下限兜底，速度控制集中在一处，避免不同难度的分支散落在主循环里。

表 3　三档难度参数表

难度	初始间隔(秒)	每级减少(秒)	间隔下限(秒)	升级所需行数
简单	0.280	0.020	0.080	15
普通	0.140	0.010	0.040	10
困难	0.093	0.007	0.027	8

```
dropInterval:
    d = 当前难度的参数
    return max(d.下限, d.初始间隔 - (等级 - 1) * d.每级减少)
```

3. 计分与等级升级

消行动画结束后，内核把本次消除的行数传给计分函数，消行数小于等于 0 时直接返回，否则每消一行加 10 分，同时把行数累计起来，等级按累计行数除以当前难度的升级行数再加 1 计算，最高分在计分后与当前分数比较取较大值；消行影响等级、等级影响下落速度，计分和难度模块由此串到一起。

```
addLines(n):
    if n <= 0: return
    score += n * 10
    totalLines += n
    level = totalLines / 升级行数 + 1
    high = max(high, score)
```

4. 排行榜与文件读写

排行榜用长度为 5 的数组保存前五名，读取时逐行读入整数，文件不存在或数据不足就把缺失位置填 0，界面始终能显示五个名次；插入新成绩时，先把旧的五个成绩和本局分数合并成六个数，整体按降序排序，再把前五个写回数组，新成绩只有排名足够靠前才会进入榜单；最高分单独存放在另一个文件里，一局结束时排行榜和最高分一起写回。

```
addScore(score):
    all = 旧的五个成绩 + 本局分数
    按降序排序 all
```

```
    取前五个写回数组
```

【此处插入截图：图 7　排行榜界面】

图 7　排行榜界面

（五）模块对接与系统集成

三个人的模块靠函数调用和状态约定对接，底层内核提供棋盘状态、方块位置和队列接口，由框架层在主循环的固定位置调用；扩展功能在框架预留的绘制点和状态切换点接入，状态切换约定为落地时发现满行就切到消行动画、动画播完再回到游戏中，出生位置不合法就切到结束界面。

集成按“先各自跑通、再逐层拼合”的顺序推进，框架先把空窗口和主循环跑起来，内核和扩展功能随后在约定的接口上并行开发；联调时先把内核挂到更新流程里验证碰撞和消行，再接入影子、难度和计分，最后检查排行榜的文件读写，每接入一层就跑一遍对应测试，问题能很快定位到具体模块。

三、调试分析

（一）底层内核问题（李鸿伟）

一是旋转踢墙卡死，早期版本只尝试原地旋转，方块靠近左右边界时旋转形态会越界，旋转表现为失效，连续按键还可能让方块停在异常位置，后来加入向左、向右各 1 到 2 格以及向上抬 1 格的候选偏移，每个候选都由 legal 验证，成功后才修改状态；

二是多行消行漏行，早期版本按找到的行号逐行删除，删掉一行后上方行下移，后续保存的行号失效，同时填满多行时会漏删，后来先用标记数组标出全部满行，再用写指针把非满行整体压实，一次性清理空出的顶部；

三是出生越界判定，出生模板有一部分位于棋盘上方，把这部分格子直接判为非法会导致正常出生失败，完全忽略边界又会掩盖顶部堆满的情况，现在只对棋盘上方的格子放行，对横向越界、底部越界和与已有方块重叠仍严格拒绝。

（二）框架交互问题（陈忠颖）

一是中文乱码，raylib 自带字体只包含 ASCII 字符，直接画中文只能看到

一排问号，最初用扩展字体加载微软雅黑的 ttc 文件时又会失败，查到库对 ttc 字体集合支持有问题，于是换成 Windows 自带的黑体 ttf，再按 UTF-8 规则手动解码出文案的全部码点，只加载实际用到的字符，字体纹理更小、加载也更快；

二是按键交互，问题集中在长按上，最初只响应按下瞬间，长按方向键方块只动一格，加入按住状态和重复计时后又出现松手后还在动，原因是按键移出窗口或在按下时切换界面、释放事件没被收到，后来在每次重开、回菜单和暂停时都主动清掉按住标志；软降和自然下落最初共用一个计时器，节奏忽快忽慢，后来软降走单步下落、自然下落单独计时，两条线互不干扰。

（三）扩展功能问题（赖霖）

一是影子位置不正确，影子判断若只看棋盘底部而忽略已固定方块，会出现影子穿过堆叠方块或停在错误位置，后来直接复用底层 legal 函数，让边界检查和碰撞检查保持一致，并且只递增临时坐标，避免影子计算改动真实方块；

二是难度速度与等级不匹配，若把速度写成多个分支，容易出现升级后某档速度没有变化或间隔变成非预期值，后来把参数集中到配置数组，由统一公式计算并用下限兜底；

三是排行榜读写和排序，首次运行时文件可能不存在或数据不足，读取函数对每次读取做判断，失败就填 0，插入成绩时把旧榜单和新分数合并后整体降序排序再截取前五，避免低分占据榜单位置。

（四）集成测试与测试数据

表 4　小组集成测试数据

组号	测试内容	操作方法	预期结果
1	菜单键盘导航	主菜单按上下键、回车、Esc	选中项上下移动，回车进入对应界面，Esc 退出
2	菜单鼠标点选	鼠标移到按钮上并点击	悬停项高亮，点击进入对应界面
3	三档难度切换	分别在简单、普通、困难开局	初始下落速度由慢到快，难度参数生效

组号	测试内容	操作方法	预期结果
4	移动长按	按住左右方向键	0.16 秒后每 0.05 秒连续移动，撞墙停住
5	软降长按	按住下方向键	方块加速连续下落，落地后正常锁定
6	边界旋转	I、L、T 形靠墙后旋转	踢墙找到合法位置，或保持原形态
7	单行消除	底部填满一行	闪烁后删除一行，分数增加 10
8	多行消除	同时填满 2 到 4 行	满行一次性删除，不漏行
9	出生检测	堆叠到顶部后生成方块	出生位置非法则进入结束界面
10	排行榜读写	删除文件后启动，再结束一局	缺失按 0 处理，新成绩按降序排入前五
11	暂停与回菜单	P 暂停，再按 Esc 回菜单	画面停住，回菜单后最高分保留
12	中文显示检查	逐一切换各界面	汉字和箭头正常显示，无问号或方块

（五）算法效率分析

棋盘规模固定为 20×16，所有核心操作的上界都比较稳定；合法性检测最多检查 16 个模板格，时间复杂度为 O(1)，落点影子最多向下试探 20 次、每次调用 legal，在固定棋盘上可视为常数；满行扫描和消行压缩各遍历一遍棋盘，时间复杂度为 O(棋盘行数 × 列数)，环形队列的入队、出队和查看队头都是 O(1)，排行榜只有 6 个元素参与排序，开销很小。

渲染上，程序固定每帧整屏重画并稳定跑满 60 帧，一帧最多画 320 个棋盘格，再加上界面文字和按钮，每个方块和矩形的绘制都是常数时间，总绘制量不随游戏时长增长；中文字体的码点收集只在启动时做一次，字体纹理一次生成、反复使用；空间方面，棋盘、方块表、队列和排行榜都是固定规模的静态存储，内存占用不随游戏时长增加。

（六）改进设想

一是按键可以做成自定义，并加入游戏手柄支持，让玩家按自己的习惯操作；

二是计分规则可以按一次消除的行数设置不同奖励，让连续消行更有成就感；

三是排行榜可以把姓名、难度和日期一起保存，分数相同时定义稳定的排序规则；

四是增强文件容错，检查分数是否为非负整数、输出文件是否成功打开，必要时保留备份；

五是界面切换可以加淡入淡出的转场动画，并适配高 DPI 屏幕；

六是把排行榜排序、等级计算和下落间隔提取成独立测试用例，减少集成修改后的回归问题。

（七）团队协作体会

这次项目让三个人对数据结构在真实程序中的位置有了更具体的认识，底层的二维数组和环形队列决定了游戏状态如何表达和供给，框架层的状态机和渲染工具决定了模块如何组织和展示，上层的计分、排行榜则把顺序表、排序和文件读写结合到了一个完整场景里，数据结构不再是课本上孤立的概念，而是各自承担着明确的职责。

协作过程中体会最深的是“接口先行”，动手写代码之前先把函数名、参数和调用时机约定清楚，三个人就能在各自的模块上并行推进，不用互相等待；模块对接时每接入一层就跑一遍测试，问题能很快定位到具体的人、具体的模块，比全部写完再一起排查要轻松很多。

分工也让每个人看清了自己的模块在整体中的位置，底层同学要保证接口稳定、框架同学要主动对接上下游、负责集成的同学要从全局检查模块之间的衔接，三个人的工作通过清晰的分层和约定拼到一起，这种协作经验和完成一个游戏本身一样有价值。

四、附录

```
// ============================================================
// 俄罗斯方块（Tetris）— 基于 raylib 图形库的单文件实现
// 功能：主菜单 / 难度选择 / 排行榜 / 暂停 / 消行动画 / 最高分存档
// 程序结构（按文件顺序）：
//   一、布局常量     二、颜色与基础绘制     三、中文字体加载
//   四、方块形状表   五、棋盘逻辑           六、难度表
//   七、排行榜       八、方块生成队列       九、游戏状态
```

```
//   十、更新逻辑     十一、绘制逻辑       十二、主程序入口
// ============================================================

#include "raylib.h"          // raylib 图形库：窗口、绘图、输入、字体等 API

#include <algorithm>           // std::clamp / std::sort / std::find / std::min /
std::max
#include <cmath>             // std::fmod（消行闪烁取余）
#include <fstream>           // std::ifstream / std::ofstream（读写存档文件）
#include <set>               // std::set（收集字体码点时自动去重 + 排序）
#include <string>            // std::string / std::to_string
#include <vector>            // std::vector（动态数组：满行列表、码点列表等）

// ============================================================
// 一、布局与尺寸常量
//    所有坐标单位都是"像素"，棋盘逻辑坐标用"格"
// ============================================================
static constexpr int BOARD_W = 16;      // 棋盘列数（横向 16 格，比经典 10 列宽）
static constexpr int BOARD_H = 20;      // 棋盘行数（纵向 20 格）
static constexpr int CELL = 30;         // 每个格子的像素边长（30×30）
static constexpr int BOARD_X = 25;      // 棋盘左上角在窗口中的 x 坐标
static constexpr int BOARD_Y = 25;      // 棋盘左上角在窗口中的 y 坐标
static constexpr int PANEL_X = 550;     // 右侧信息栏（分数/预览）的起点 x
static constexpr int WIN_W = 800;       // 窗口宽度（像素）
static constexpr int WIN_H = 650;       // 窗口高度（像素）

static constexpr int RANK_SIZE = 5;     // 排行榜最多保存 / 显示的条数
static constexpr const char* SCORE_FILE = "high_score.txt";  // 最高分存档文件名
static constexpr const char* RANK_FILE = "leaderboard.txt";  // 排行榜存档文件名

// ============================================================
// 二、颜色（RGBA）
//    raylib 的 Color 结构体含 r/g/b/a 四个分量，范围 0~255
// ============================================================
static const Color kBg      = {18, 18, 24, 255};       //很深的蓝灰色，整个窗口的底色
static const Color kBoardBg  = {11, 11, 16, 255};       //比底色更暗的近黑色，棋盘内部
的背景
static const Color kGrid     = {30, 30, 40, 255};       //很暗的灰蓝，棋盘上画格子用的
细线
static const Color kText     = {226, 226, 232, 255};    //接近纯白的亮灰，主要正文和数
字
static const Color kTextDim  = {128, 128, 144, 255};    //中灰色，次要提示/说明文字
static const Color kGold     = {240, 200, 70, 255};     //金黄色，强调色
static const Color kSelect   = {52, 84, 158, 255};      //中等蓝色，菜单按钮被选中时的
```

高亮底色

```
// 形状编号：0=T 1=J 2=L 3=Z 4=S 5=O 6=I，下标直接对应颜色
// 也就是说：方块是哪种形状，就用 kShapeColor[形状编号] 取它的颜色
static const Color kShapeColor[7] = {
    {176, 80, 225, 255},   // T 型 紫色
    {52, 92, 232, 255},    // J 型 蓝色
    {238, 142, 38, 255},   // L 型 橙色
    {60, 185, 80, 255},    // Z 型 绿色
    {228, 68, 72, 255},    // S 型 红色
    {232, 205, 55, 255},   // O 型 黄色
    {45, 198, 218, 255}    // I 型 青色
};

// 把颜色按倍数 f 整体调亮或调暗（用于给方块画高光和暗边，制造立体感）
static Color Shade(Color c, float f)    // 参数 c：原色；f：倍率，f>1.0 调亮，f<1.0 调暗
{
    return Color{
        // 每个分量先乘倍率，再用 clamp 限制在 0~255 之间，最后转回无符号字节
        (unsigned char)std::clamp((int)(c.r * f), 0, 255),   // 红通道
        (unsigned char)std::clamp((int)(c.g * f), 0, 255),   // 绿通道
        (unsigned char)std::clamp((int)(c.b * f), 0, 255),   // 蓝通道
        c.a};                                                // 透明度不变
}

// 画一个带立体高光的小方块（棋盘里每一格、活动方块、预览方块都用它）
static void DrawCell(int x, int y, int sz, Color c)  // (x,y)：左上角像素坐标；sz：边长；c：主色
{
    DrawRectangle(x, y, sz, sz, c);     // 先画方块主体（纯色正方形）
    int b = sz >= 25 ? 3 : 2;           // 高光/暗边的宽度：大格子画 3 像素，小格子画 2 像素
    DrawRectangle(x + 1, y + 1, sz - 2, b, Shade(c, 1.35f));            // 顶部高光（调亮）
    DrawRectangle(x + 1, y + 1, b, sz - 2, Shade(c, 1.35f));            // 左侧高光（调亮）
    DrawRectangle(x + 1, y + sz - 1 - b, sz - 2, b, Shade(c, 0.55f));   // 底部暗边（调暗）
    DrawRectangle(x + sz - 1 - b, y + 1, b, sz - 2, Shade(c, 0.55f));   // 右侧暗边（调暗）
}

// 画落点影子（Ghost Piece）：提示方块如果直接落下去会停在哪里
```

```
static void DrawGhostCell(int x, int y, int sz, Color c)  // 参数含义同 DrawCell
{
    // 第一层：半透明填充（alpha=40，几乎只透出一点颜色）
    DrawRectangle(x + 1, y + 1, sz - 2, sz - 2, Color{c.r, c.g, c.b, 40});
    // 第二层：半透明描边（alpha=150，比填充更明显，勾勒出方块轮廓）
    DrawRectangleLinesEx(Rectangle{x + 1.0f, y + 1.0f, sz - 2.0f, sz - 2.0f},
                         1.5f, Color{c.r, c.g, c.b, 150});
}

// ============================================================
// 三、中文字体加载
//    raylib 默认字体不含汉字，需要用 LoadFontEx 只加载界面
//    用到的那些字符（码点），既省内存又能正常显示中文
// ============================================================
// 所有界面会用到的中文文案都集中列在这里，
// 程序启动时从这些字符串里提取汉字码点，拼成字体加载清单
static const char* kUiTexts[] = {
    "俄罗斯方块",
    "开始游戏", "排行榜", "退出游戏",
    "↑↓ 选择，回车确认，也可点击鼠标",
    "选择难度", "模式", "简单", "普通", "困难",
    "适合新手，下落较慢", "标准速度", "高速挑战",
    "↑↓ 选择，回车确认，Esc 返回",
    "第 ", " 名    ", " 分", "---",
    "按任意键或点击鼠标返回",
    "下一个", "当前分数", "最高纪录", "当前等级", "消除行数",
    "← → 移动", "↓ 加速", "↑/空格 旋转", "P 暂停", "R 重开", "Esc 菜单",
    "难度：",
    "游戏暂停", "按 P 继续", "Esc 返回主菜单",
    "游戏结束", "本局得分：", "成绩已记入排行榜",
    "R 重新开始", "Esc 返回主菜单"};

// 从 kUiTexts 里解析出所有字符的 Unicode 码点（含 ASCII、箭头和汉字）
// 返回的码点列表经过 set 自动去重并升序排列，供 LoadFontEx 使用
static std::vector<int> CollectCodePoints()
{
    std::set<int> s;                              // 用 set 存码点：自动去重、自动排序
    for (int c = 32; c <= 126; ++c) s.insert(c);  // 先塞入全部 ASCII 可打印字符（空格~波浪号）
    static const int extra[] = {
        0x2190, 0x2191, 0x2192, 0x2193,     // ← ↑ → ↓ 四个方向箭头的 Unicode 码点
        0xFF0C, 0xFF1A,                    // 中文全角逗号"，"和全角冒号"："
        0x2014};                           // 中文破折号"—"
    for (int c : extra) s.insert(c);         // 把这些特殊符号也加入集合
```

```
    // 手动按 UTF-8 编码规则解析每个字符串中汉字的码点
    // （源码文件以 UTF-8 保存，每个汉字在内存里占 3 个字节）
    for (const char* t : kUiTexts)
    {
        const unsigned char* p = (const unsigned char*)t;  // 按无符号字节逐个读取
        while (*p)                                          // 一直读到字符串结尾的 '\0'
        {
            int cp = 0;                                     // cp：解析出的 Unicode 码点
            if (*p < 0x80) { cp = *p; ++p; }    // 0xxxxxxx：单字节 ASCII，码点就是它本身
            else if ((*p & 0xE0) == 0xC0)       // 110xxxxx：2 字节字符（本次实际用不到）
            {
                // 取第 1 字节低 5 位、第 2 字节低 6 位拼接
                cp = ((p[0] & 0x1F) << 6) | (p[1] & 0x3F);
                p += 2;                                     // 跳过这 2 个字节
            }
            else if ((*p & 0xF0) == 0xE0)       // 1110xxxx：3 字节字符（绝大多数汉字）
            {
                // 取第 1 字节低 4 位、第 2 字节低 6 位、第 3 字节低 6 位拼接
                cp = ((p[0] & 0x0F) << 12) | ((p[1] & 0x3F) << 6) | (p[2] & 0x3F);
                p += 3;                                     // 跳过这 3 个字节
            }
            else if ((*p & 0xF8) == 0xF0)       // 11110xxx：4 字节字符（emoji 等，极少用到）
            {
                // 4 个字节各取有效位拼接
                cp = ((p[0] & 0x07) << 18) | ((p[1] & 0x3F) << 12) |
                     ((p[2] & 0x3F) << 6) | (p[3] & 0x3F);
                p += 4;                                     // 跳过这 4 个字节
            }
            else ++p;                           // 异常字节：直接跳过，避免死循环
            s.insert(cp);                           // 把解析出的码点放进集合去重
        }
    }
    return std::vector<int>(s.begin(), s.end());  // 把 set 转成 vector 返回（已升序）
}

static Font fontUI;   // 普通字号字体（界面正文、数字）
static Font fontBig;  // 大字号字体（标题）

// 水平居中绘制一段文字
static void TextCenter(Font f, const char* text, float cx, float y, float size,
```

```
Color c)
// 参数：f 字体；text 文字；cx 中心点 x；y 顶部 y；size 字号；c 颜色
{
    Vector2 m = MeasureTextEx(f, text, size, 1.0f);                // 先测量文字的宽和高
    DrawTextEx(f, text, Vector2{cx - m.x / 2, y}, size, 1.0f, c);   // 左边界 = 中心 - 半个字宽
}

// ============================================================
// 四、方块形状表（四维数组 piece[7][4][4][4]）
//    四个下标的含义：piece[形状编号][旋转形态][行][列]
//    值为 1 表示该格有方块，0 表示空
//    每种形状都有 4 种旋转形态（0/1/2/3），在 4×4 小矩阵里描述
// ============================================================
static int g_piece[7][4][4][4];

// 初始化全部 28 个方块矩阵（7 种形状 × 4 种旋转）：
// 先手工填好每种形状的 0 号形态，再用旋转公式批量生成其余 3 种
static void InitPieces()
{
    // 第一步：把所有格子清零，防止残留随机值
    for (int sh = 0; sh < 7; ++sh)        // 遍历 7 种形状
        for (int f = 0; f < 4; ++f)       // 遍历 4 种旋转形态
            for (int i = 0; i < 4; ++i)   // 遍历行
                for (int j = 0; j < 4; ++j)  // 遍历列
                    g_piece[sh][f][i][j] = 0;  // 置空

    // T 形（0 号）：第一行横放 3 格，第二行中间补 1 格
    //   [ ][ ][ ]
    //   [X][X][X]      （第 1 行，列 0~2）
    //   [ ][X]         （第 2 行，列 1）
    for (int i = 0; i <= 2; ++i) g_piece[0][0][1][i] = 1;  // 横杠 3 格
    g_piece[0][0][2][1] = 1;                               // 中间凸起 1 格

    // J 形（1 号）和 L 形（2 号）：都是竖 3 格 + 底部一个拐弯
    for (int i = 1; i <= 3; ++i)
    {
        g_piece[1][0][i][1] = 1;   // J：竖杠在列 1
        g_piece[2][0][i][2] = 1;   // L：竖杠在列 2（镜像）
    }
    g_piece[1][0][3][2] = 1;       // J：底部向右拐一格
    g_piece[2][0][3][1] = 1;       // L：底部向左拐一格
```

```
    // Z（3 号）、S（4 号）、O（5 号）：都由两行、每行两格错开组成
    for (int i = 0; i <= 1; ++i)
    {
        g_piece[3][0][1][i] = 1;        // Z 上行：列 0、1
        g_piece[3][0][2][i + 1] = 1;    // Z 下行：列 1、2（向右错开）
        g_piece[4][0][1][i + 1] = 1;    // S 上行：列 1、2
        g_piece[4][0][2][i] = 1;        // S 下行：列 0、1（向左错开，与 Z 镜像）
        g_piece[5][0][1][i + 1] = 1;    // O 上行：列 1、2
        g_piece[5][0][2][i + 1] = 1;    // O 下行：列 1、2（上下对齐，2×2 方块）
    }

    // I 形（6 号）：一条横杠 4 格
    for (int i = 0; i <= 3; ++i) g_piece[6][0][1][i] = 1;  // 第 1 行横放 4 格

    // 第二步：由 0 号形态顺时针旋转 90° 生成 1 号，再依次生成 2、3 号
    // 旋转公式：新矩阵[i][j] = 旧矩阵[3-j][i]
    // （把 4×4 矩阵顺时针转 90°：原"行"变成新"列"，顺序倒置）
    for (int sh = 0; sh < 7; ++sh)        // 每种形状
        for (int f = 0; f < 3; ++f)       // 由形态 f 推出形态 f+1，共推 3 次
            for (int i = 0; i < 4; ++i)   // 新矩阵行
                for (int j = 0; j < 4; ++j)  // 新矩阵列
                    g_piece[sh][f + 1][i][j] = g_piece[sh][f][3 - j][i];
}

// ============================================================
// 五、棋盘（二维数组 cell[20][16]）
//    cell[y][x] 取值：
//      -1 表示该格为空；
//      0~6 表示该格被对应形状编号的方块占据（用于取颜色）
// ============================================================
static int cell[BOARD_H][BOARD_W];

// 清空棋盘：把所有格子置为 -1（开局 / 重开时调用）
static void resetBoard()
{
    for (int i = 0; i < BOARD_H; ++i)       // 棋盘行遍历，对应棋盘高
        for (int j = 0; j < BOARD_W; ++j)   // 棋盘列遍历，对应棋盘宽
            cell[i][j] = -1;                // 置空
}

// 判断方块放在棋盘坐标 (bx, by) 处是否合法：
// 条件是不越左右下边界、不与已锁定的方块重叠
// 特殊处理：by < 0（方块在棋盘上方、部分在屏幕外）时放行，
// 因为方块刚从顶部出生时本来就有一半在屏外
```

```
static bool legal(int shape, int form, int bx, int by)
// shape 形状编号；form 旋转形态；(bx,by) 方块 4×4 矩阵左上角对应的棋盘格坐标
{
    for (int i = 0; i < 4; ++i)           // 遍历方块矩阵的行
    {
        for (int j = 0; j < 4; ++j)       // 遍历方块矩阵的列
        {
            if (g_piece[shape][form][i][j] != 1) continue;  // 该格为空，跳过
            int x = bx + j;               // 该格对应的棋盘列
            int y = by + i;               // 该格对应的棋盘行
            if (x < 0 || x >= BOARD_W || y >= BOARD_H) return false;  // 越左右/下边
界，非法
            if (y < 0) continue;          // 在棋盘上方（出生区），暂不判定
            if (cell[y][x] != -1) return false;  // 该格已有方块，碰撞，非法
        }
    }
    return true;                          // 所有格子都通过检查，合法
}

// 把活动方块写进棋盘（方块落地锁定时调用）：
// 把它占据的每个格子标记为它的形状编号，之后这些格子就固定下来
static void lockPiece(int shape, int form, int bx, int by)
{
    for (int i = 0; i < 4; ++i)           // 遍历方块矩阵行
        for (int j = 0; j < 4; ++j)       // 遍历方块矩阵列
            if (g_piece[shape][form][i][j] == 1)   // 该格有方块
            {
                int y = by + i, x = bx + j;        // 换算成棋盘坐标
                // 写入前再做一次边界保护（屏外的格子不写）
                if (y >= 0 && y < BOARD_H && x >= 0 && x < BOARD_W)
                    cell[y][x] = shape;    // 记录形状编号（决定颜色）
            }
}

// 找出棋盘上所有"已经填满"的行，返回这些行的行号列表
static std::vector<int> fullRows()
{
    std::vector<int> rows;                // 存放满行的行号
    for (int y = 0; y < BOARD_H; ++y)     // 逐行检查
    {
        bool full = true;                 // 先假设这一行是满的
        for (int x = 0; x < BOARD_W; ++x) // 检查这一行的每一格
            if (cell[y][x] == -1) { full = false; break; }  // 只要有一个空格，就不是
满行
```

```
        if (full) rows.push_back(y);       // 整行无空格，记录行号
    }
    return rows;
}

// 删除指定的满行（消行）：
// 思路是"从下往上扫描 + 写指针"，把非满行整体向下压实，顶部空出来的行补空
// 这样即使连续多行同时填满，也能一次性正确处理，不会漏删
static void removeRows(const std::vector<int>& rows)
{
    std::vector<bool> full(BOARD_H, false);  // 标记每一行是否为要删除的满行
    for (int r : rows) full[r] = true;       // 把传入的行号标记为 true

    int write = BOARD_H - 1;                 // write："下一个保留行应该写到哪一行"，从
底部开始
    for (int y = BOARD_H - 1; y >= 0; --y)  // 从最底行向上扫描
    {
        if (full[y]) continue;               // 满行：跳过（相当于删除，不往下写）
        for (int x = 0; x < BOARD_W; ++x)   // 非满行：整行复制到 write 指向的位置
            cell[write][x] = cell[y][x];
        --write;                             // 写指针上移，准备放下一个保留行
    }
    // 扫描结束后，write 以上（含）都是空出来的位置，逐行补 -1
    for (; write >= 0; --write)
        for (int x = 0; x < BOARD_W; ++x)
            cell[write][x] = -1;
}

// ============================================================
// 六、难度表（结构体数组）
//    方块下落速度由"下落间隔（秒）"控制：间隔越短，下落越快
// ============================================================
struct Difficulty
{
    const char* name;       // 难度名（简单 / 普通 / 困难）
    const char* desc;       // 一句话说明（菜单里显示）
    float initial;          // 第 1 级时的初始下落间隔（秒）
    float step;            // 每升一级，下落间隔减少多少（秒）
    float minimum;          // 下落间隔的下限（再升级也不会比这更快）
    int linesPerLevel;      // 每消除多少行升一级
};

// 三种难度的具体数值表
static const Difficulty kDiff[3] = {
```

```
    //name  desc                initial  step    minimum  linesPerLevel
    {"简单", "适合新手，下落较慢", 0.280f, 0.020f, 0.080f, 15},  // 起步慢、升级慢
    {"普通", "标准速度",           0.140f, 0.010f, 0.040f, 10},  // 标准体验
    {"困难", "高速挑战",           0.093f, 0.007f, 0.027f, 8}};  // 起步就很快

// ============================================================
// 七、排行榜（数组 scores[5] + 排序）
//    分数从高到低保存前 5 名，持久化到 leaderboard.txt
// ============================================================
static int scores[RANK_SIZE];   // 排行榜数据，scores[0] 为第 1 名

// 从文件读取排行榜；文件不存在或行数不足时，缺失位置填 0
static void loadRanks()
{
    std::ifstream in(RANK_FILE);                // 打开排行榜文件
    for (int i = 0; i < RANK_SIZE; ++i)          // 依次读 5 个数
        if (!(in >> scores[i])) scores[i] = 0;    // 读失败（文件结束/不存在）就填 0
}

// 把当前排行榜写回文件（每行一个分数）
static void saveRanks()
{
    std::ofstream out(RANK_FILE);               // 以覆盖方式打开文件
    for (int i = 0; i < RANK_SIZE; ++i) out << scores[i] << '\n';  // 逐行写入
}

// 插入一个新分数：把新分数和旧的 5 个合并，降序排序后只保留前 5 名
static void addScore(int score)
{
    std::vector<int> all;                       // 临时存放"旧分数 + 新分数"
    for (int i = 0; i < RANK_SIZE; ++i) all.push_back(scores[i]);  // 装入旧的 5 个
    all.push_back(score);                       // 装入本局新分数（共 6 个）
    std::sort(all.begin(), all.end(), std::greater<int>());  // 降序排序（大的在前）
    for (int i = 0; i < RANK_SIZE; ++i) scores[i] = all[i];  // 取前 5 名写回
}

// ============================================================
// 八、方块生成队列（数组实现的环形队列 / FIFO）
//    作用：提前随机生成若干方块排好队，按顺序依次出现，
//    这样右侧"下一个"预览能提前展示接下来的方块。
//    队头取走一个，立刻在队尾补一个随机块，队列长度保持不变。
// ============================================================
struct PiecePair
{
```

```
    int shape;   // 形状编号（0~6）
    int form;    // 初始旋转形态（0~3）
};

static const int Q_CAP = 8;           // 环形队列容量（最多放 8 个）
static PiecePair pieceQueue[Q_CAP];   // 队列的实际存储空间
static int qHead = 0;                 // 队头下标（下一个要取出的位置）
static int qTail = 0;                 // 队尾下标（下一个要放入的位置）
static int qCount = 0;                // 队列中当前的元素个数

// 清空队列并预填 3 个随机方块（游戏开始时调用）
static void qInit()
{
    qHead = qTail = qCount = 0;               // 三个下标 / 计数全部归零
    for (int i = 0; i < 3; ++i)               // 预填 3 个
    {
        // 随机形状 0~6、随机初始形态 0~3，写入队尾
        pieceQueue[qTail] = {GetRandomValue(0, 6), GetRandomValue(0, 3)};
        qTail = (qTail + 1) % Q_CAP;          // 队尾后移，到末尾则绕回 0（环形）
        ++qCount;                             // 元素数 +1
    }
}

// 在队尾入队一个随机方块
static void qPushRandom()
{
    pieceQueue[qTail] = {GetRandomValue(0, 6), GetRandomValue(0, 3)};  // 生成随机块
写入队尾
    qTail = (qTail + 1) % Q_CAP;              // 队尾后移并取模，实现环形
    ++qCount;                                 // 元素数 +1
}

// 从队头取出一个方块并返回，同时在队尾补一个随机块（保持队列长度）
static PiecePair qPop()
{
    PiecePair p = pieceQueue[qHead];          // 取队头方块
    qHead = (qHead + 1) % Q_CAP;              // 队头后移并取模
    --qCount;                                 // 元素数 -1
    qPushRandom();                            // 立刻补一个随机块，队列重新填满
    return p;                                 // 返回取出的方块
}

// 只查看队头方块（即"下一个"会出现的方块），但不取出
static PiecePair qFront()
```

```
{
    return pieceQueue[qHead];                   // 直接返回队头元素
}

// ============================================================
// 九、游戏状态（全局变量）
//    游戏用一个"状态机"在各界面之间切换，phase 记录当前状态
// ============================================================
enum Phase
{
    PH_MENU,        // 主菜单
    PH_DIFFICULTY,  // 难度选择
    PH_RANK,        // 排行榜展示
    PH_PLAYING,     // 游戏进行中
    PH_PAUSED,      // 暂停
    PH_CLEARING,    // 消行闪烁动画播放中（此时暂不接受操作）
    PH_GAMEOVER     // 游戏结束
};

// 前向声明：下面这些函数之间存在互相调用，
// 先在这里声明签名，后面再给出具体定义，编译器才不会报错
static void spawn();
static void endGame();
static void updateMenu();
static void chooseMenu(int i);
static void updateDifficulty();
static void updateRank();
static void updatePaused();
static void updateOver();
static void updatePlaying(float dt);
static void updateClearing(float dt);
static void drawMenu();
static void drawDifficulty();
static void drawRank();
static void drawGame();
static void drawPause();
static void drawGameOverScreen();
static void drawPanel();
static void statBlock(float x, int y, const char* label, int value);
static void drawNext(Rectangle pv);

static Phase phase = PH_MENU;   // 当前所处的界面状态，启动时进入主菜单
static bool wantExit = false;   // 是否请求退出程序（菜单选"退出"时置 true）
```

```
static int menuSel = 0;   // 主菜单当前选中项（0 开始 / 1 排行榜 / 2 退出）
static int diffSel = 1;   // 难度当前选中项（默认 1 = 普通）

// 正在下落的活动方块的信息
static int shape = 0, form = 0, px = 3, py = -1;
// shape：形状编号；form：当前旋转形态；
// px,py：方块 4×4 矩阵左上角在棋盘中的格坐标（py 初始 -1，从顶部出生）
static int diffIdx = 1;   // 当前对局使用的难度下标

// 计分相关
static int level = 1;        // 当前等级（随消行提升，等级越高下落越快）
static int totalLines = 0;   // 累计消除的行数
static int score = 0;        // 当前分数
static int high = 0;         // 历史最高分

// 计时器（单位：秒）
static float dropTimer = 0.0f;   // 自然下落累计时间，达到下落间隔就下落一格
static float repL = 0.0f, repR = 0.0f, repD = 0.0f;  // 左/右/下三键"长按重复"的累计计时
static bool holdL = false, holdR = false, holdD = false;  // 三个方向键当前是否处于按住状态

// 消行动画相关
static std::vector<int> clearing;  // 正在闪烁、即将被删除的行号列表
static float clearTimer = 0.0f;    // 消行动画已播放的时间

// 菜单 / 难度按钮的可点击矩形区域（绘制时记录，鼠标点击时做命中检测）
static Rectangle menuBounds[3];
static Rectangle diffBounds[3];

// 计算当前等级下的下落间隔（秒）：
// 初始间隔 - (等级-1)×每级缩短量，但不会低于下限
static float dropInterval()
{
    const Difficulty& d = kDiff[diffIdx];             // 取当前难度的参数
    return std::max(d.minimum, d.initial - (level - 1) * d.step);  // 等级越高间隔越小，下限兜底
}

// 把最高分保存到文件
static void saveHigh()
{
    std::ofstream out(SCORE_FILE);      // 打开最高分文件（覆盖写）
    if (out) out << high;              // 文件正常打开才写入
```

```
}

// 开始一局新游戏：重置棋盘和所有计分数据、初始化方块队列、生成第一个方块
static void startGame(int idx)
{
    diffIdx = idx;              // 记录本局难度
    resetBoard();               // 清空棋盘
    score = 0;                  // 分数清零
    level = 1;                  // 等级回到 1
    totalLines = 0;             // 消行数清零
    dropTimer = 0.0f;           // 下落计时器清零
    holdL = holdR = holdD = false;  // 松开所有方向键的按住标记
    repL = repR = repD = 0.0f;  // 长按重复计时器清零
    clearing.clear();           // 清空可能残留的消行列表
    qInit();                    // 初始化方块队列（预填 3 个随机块）
    spawn();                    // 生成第一个活动方块
    phase = PH_PLAYING;         // 切换到游戏进行中
}

// 从队列队头取出下一个方块，放到棋盘顶部准备下落；
// 如果一出生位置就不合法（方块堆到了顶部），判定游戏结束
static void spawn()
{
    PiecePair cur = qPop();     // 从队列取出下一个方块
    shape = cur.shape;          // 设置形状
    form = cur.form;            // 设置初始旋转形态
    px = 3;                     // 出生列（水平居中偏左）
    py = -1;                    // 出生行：-1，让方块从棋盘上方逐渐进入
    dropTimer = 0.0f;           // 重置下落计时器
    if (!legal(shape, form, px, py)) endGame();  // 出生即碰撞，说明堆满，结束游戏
}

// 游戏结束处理：本局分数写入排行榜、更新最高分、切换到结束界面
static void endGame()
{
    addScore(score);            // 本局分数并入排行榜
    saveRanks();                // 排行榜写回文件
    high = std::max(high, score);  // 更新历史最高分
    saveHigh();                 // 最高分写回文件
    phase = PH_GAMEOVER;        // 切换到游戏结束界面
}

// 尝试把活动方块平移 (dx, dy) 格：
// 目标位置合法就移动，返回 true；不合法（撞墙/撞块）则原地不动，返回 false
```

```
static bool moveBy(int dx, int dy)
{
    if (!legal(shape, form, px + dx, py + dy)) return false;  // 先试探目标位置
    px += dx;                  // 合法：更新列坐标
    py += dy;                  // 更新行坐标
    return true;               // 移动成功
}

// 旋转方块：先尝试原地旋转，若被挡住则做"踢墙（Wall Kick）"，
// 即尝试向左右平移、或向上抬一格后再旋转，尽量让旋转成功
static void rotate()
{
    int nf = (form + 1) % 4;    // 旋转后的新形态编号（顺时针，形态 3 之后回到 0）
    static const int ox[] = {0, -1, 1, -2, 2};  // 水平方向依次尝试：原位、左 1、右 1、
左 2、右 2
    for (int oy = 0; oy >= -1; --oy)           // 垂直方向尝试：先原位，再上抬 1 格
        for (int k = 0; k < 5; ++k)            // 依次尝试 5 种水平偏移
            if (legal(shape, nf, px + ox[k], py + oy))  // 该偏移下旋转合法
            {
                px += ox[k];    // 应用水平偏移
                py += oy;       // 应用垂直偏移
                form = nf;      // 确认旋转到新形态
                return;         // 旋转成功，结束
            }
    // 所有偏移都不合法：放弃本次旋转，方块保持原样
}

// 方块落地锁定：先把方块写进棋盘，再检查是否有满行；
// 有满行就进入消行动画状态，没有满行就直接生成下一个方块
static void beginLock()
{
    lockPiece(shape, form, px, py);  // 把活动方块固定到棋盘上
    std::vector<int> rows = fullRows();  // 检查是否出现满行
    if (!rows.empty())
    {
        clearing = rows;          // 记录要闪烁消除的行
        clearTimer = 0.0f;        // 动画计时器清零
        phase = PH_CLEARING;      // 进入消行动画状态
    }
    else spawn();                 // 没有满行，直接出下一个方块
}

// 让方块向下走一格；如果向下走不动（到底或撞到方块），就落地锁定
static void stepDown()
```

```
{
    if (!moveBy(0, 1)) beginLock();  // 试探下移 1 格，失败则落地
}

// 计算影子方块的行位置：从当前行开始不断向下试探，直到再往下就不合法为止
static int shadowY()
{
    int sy = py;                          // 从方块当前行开始
    while (legal(shape, form, px, sy + 1)) ++sy;  // 下一格合法就继续下移
    return sy;                            // 返回最终能落到的行
}

// 消除 n 行后更新分数、累计行数和等级
static void addLines(int n)
{
    if (n <= 0) return;           // 没消行不处理
    score += n * 10;              // 每消 1 行得 10 分
    totalLines += n;              // 累计消行数
    // 等级 = 累计行数 / 每级所需行数 + 1（整数除法，达到指定行数才升级）
    level = totalLines / kDiff[diffIdx].linesPerLevel + 1;
    high = std::max(high, score);  // 顺手刷新最高分
}

// ============================================================
// 十、更新逻辑（每帧调用一次，按当前状态分发到对应处理函数）
//    dt 是"上一帧到这一帧经过的时间（秒）"，用于计时器
// ============================================================
static void update(float dt)
{
    switch (phase)
    {
    case PH_MENU:       updateMenu(); break;        // 主菜单
    case PH_DIFFICULTY: updateDifficulty(); break;  // 难度选择
    case PH_RANK:       updateRank(); break;        // 排行榜
    case PH_PLAYING:    updatePlaying(dt); break;   // 游戏中（需要 dt）
    case PH_PAUSED:     updatePaused(); break;      // 暂停
    case PH_CLEARING:   updateClearing(dt); break;  // 消行动画（需要 dt）
    case PH_GAMEOVER:   updateOver(); break;        // 游戏结束
    }
}

// 主菜单的交互：
// 鼠标悬停自动切换选中项；↑↓ 键移动选择；回车确认；Esc 退出；鼠标点击直接选中
static void updateMenu()
```

```
{
    Vector2 m = GetMousePosition();                // 获取当前鼠标坐标
    for (int i = 0; i < 3; ++i)                    // 检查鼠标是否悬停在某个按钮上
        if (CheckCollisionPointRec(m, menuBounds[i])) menuSel = i;  // 悬停则把该项设为选中
    if (IsKeyPressed(KEY_UP) && menuSel > 0) --menuSel;    // ↑：选中项上移（不超过 0）
    if (IsKeyPressed(KEY_DOWN) && menuSel < 2) ++menuSel;  // ↓：选中项下移（不超过 2）
    if (IsKeyPressed(KEY_ENTER)) chooseMenu(menuSel);      // 回车：确认当前选中项
    if (IsKeyPressed(KEY_ESCAPE)) wantExit = true;         // Esc：退出程序
    if (IsMouseButtonPressed(MOUSE_BUTTON_LEFT))           // 鼠标左键点击
        for (int i = 0; i < 3; ++i)
            if (CheckCollisionPointRec(m, menuBounds[i])) chooseMenu(i);  // 点中哪个就执行哪个
}

// 菜单某个选项被确认后执行的动作
static void chooseMenu(int i)
{
    if (i == 0) phase = PH_DIFFICULTY;   // 0：开始游戏 → 先进入难度选择
    else if (i == 1) phase = PH_RANK;    // 1：查看排行榜
    else wantExit = true;                // 2：退出游戏
}

// 难度选择界面的交互（逻辑与主菜单类似）
static void updateDifficulty()
{
    Vector2 m = GetMousePosition();                 // 鼠标坐标
    for (int i = 0; i < 3; ++i)                     // 悬停切换选中
        if (CheckCollisionPointRec(m, diffBounds[i])) diffSel = i;
    if (IsKeyPressed(KEY_UP) && diffSel > 0) --diffSel;    // ↑ 上移
    if (IsKeyPressed(KEY_DOWN) && diffSel < 2) ++diffSel;  // ↓ 下移
    if (IsKeyPressed(KEY_ENTER)) startGame(diffSel);       // 回车：按所选难度开局
    if (IsKeyPressed(KEY_ESCAPE)) phase = PH_MENU;         // Esc：返回主菜单
    if (IsMouseButtonPressed(MOUSE_BUTTON_LEFT))           // 鼠标点击
        for (int i = 0; i < 3; ++i)
            if (CheckCollisionPointRec(m, diffBounds[i])) startGame(i);  // 点中即按该难度开局
}

// 排行榜界面：按任意键或点击鼠标，返回主菜单
static void updateRank()
{
    if (GetKeyPressed() != 0 || IsMouseButtonPressed(MOUSE_BUTTON_LEFT))
        phase = PH_MENU;
```

```
}

// 暂停界面：按 P 或 S 继续游戏；按 Esc 保存最高分并返回主菜单
static void updatePaused()
{
    if (IsKeyPressed(KEY_P) || IsKeyPressed(KEY_S)) phase = PH_PLAYING;  // 继续
    if (IsKeyPressed(KEY_ESCAPE))
    {
        saveHigh();            // 回主菜单前先保存最高分
        phase = PH_MENU;
    }
}

// 游戏结束界面：R 用同一难度重新开始；回车 / Esc / 点击鼠标返回主菜单
static void updateOver()
{
    if (IsKeyPressed(KEY_R)) startGame(diffIdx);                        // 重新开始
    if (IsKeyPressed(KEY_ENTER) || IsKeyPressed(KEY_ESCAPE)) phase = PH_MENU;  // 回
菜单
    if (IsMouseButtonPressed(MOUSE_BUTTON_LEFT)) phase = PH_MENU;      // 点击回菜单
}

// 游戏进行中的核心交互：处理退出/暂停/重开、左右移动、软降、旋转、自动下落
static void updatePlaying(float dt)
{
    // 全局类按键（触发后直接切换状态，本帧不再处理其他逻辑）
    if (IsKeyPressed(KEY_ESCAPE)) { saveHigh(); phase = PH_MENU; return; }  // Esc
回菜单
    if (IsKeyPressed(KEY_P) || IsKeyPressed(KEY_S)) { phase = PH_PAUSED; return; }
// P/S 暂停
    if (IsKeyPressed(KEY_R)) { startGame(diffIdx); return; }  // R 重新开始

    // ---- 左移（带"长按自动重复"：按住不放会连续移动）----
    if (IsKeyPressed(KEY_LEFT)) { moveBy(-1, 0); holdL = true; repL = 0.0f; }  // 按
下瞬间先移一格
    if (IsKeyReleased(KEY_LEFT)) holdL = false;   // 松开：清除按住标记
    if (holdL && IsKeyDown(KEY_LEFT))            // 仍在按住
    {
        repL += dt;                              // 累计按住时间
        if (repL >= 0.16f) { moveBy(-1, 0); repL -= 0.05f; }  // 超过 0.16 秒后，每
0.05 秒移一格
    }

    // ---- 右移（逻辑与左移相同，方向相反）----
```

```
    if (IsKeyPressed(KEY_RIGHT)) { moveBy(1, 0); holdR = true; repR = 0.0f; }
    if (IsKeyReleased(KEY_RIGHT)) holdR = false;
    if (holdR && IsKeyDown(KEY_RIGHT))
    {
        repR += dt;
        if (repR >= 0.16f) { moveBy(1, 0); repR -= 0.05f; }
    }

    // ---- 软降（按住 ↓ 加速下落）----
    if (IsKeyPressed(KEY_DOWN)) { stepDown(); holdD = true; repD = 0.0f; }  // 按下瞬间先降一格
    if (IsKeyReleased(KEY_DOWN)) holdD = false;
    if (holdD && IsKeyDown(KEY_DOWN))
    {
        repD += dt;
        if (repD >= 0.05f) { stepDown(); repD -= 0.05f; }  // 按住时每 0.05 秒降一格（比左右快）
    }

    // ---- 旋转：↑ 或 空格 ----
    if (IsKeyPressed(KEY_UP) || IsKeyPressed(KEY_SPACE)) rotate();

    if (phase != PH_PLAYING) return;   // 若上面的操作触发了落地/结束（状态已变），本帧不再下落

    // ---- 自然下落（重力）：累计时间达到当前等级的下落间隔，就自动降一格 ----
    dropTimer += dt;                          // 累计本帧时间
    if (dropTimer >= dropInterval())          // 达到下落间隔
    {
        dropTimer = 0.0f;                     // 计时器清零
        stepDown();                           // 下落一格（走不动则落地）
    }
}

// 消行动画更新：闪烁动画播满 0.42 秒后，真正删除满行、结算分数、生成下一个方块
static void updateClearing(float dt)
{
    clearTimer += dt;                 // 累计动画时间
    if (clearTimer >= 0.42f)          // 动画时长到
    {
        removeRows(clearing);         // 真正删除满行、压实上方方块
        addLines((int)clearing.size());  // 按消除行数结算分数和等级
        clearing.clear();             // 清空消行列表
        spawn();                      // 生成下一个方块
```

```
        if (phase == PH_CLEARING) phase = PH_PLAYING;  // 恢复游戏进行状态
    }
}

// ============================================================
// 十一、绘制（每帧调用一次，把当前画面画到屏幕上）
// ============================================================

// 绘制总入口：每帧先铺一层背景底色，再根据当前状态画对应界面
static void draw()
{
    DrawRectangle(0, 0, WIN_W, WIN_H, kBg);   // 用背景色铺满整个窗口（同时清掉上一帧）
    switch (phase)
    {
    case PH_MENU: drawMenu(); break;          // 主菜单
    case PH_DIFFICULTY: drawDifficulty(); break;  // 难度选择
    case PH_RANK: drawRank(); break;          // 排行榜
    default:
        // 游戏中 / 暂停 / 消行 / 结束 都以游戏画面为底
        drawGame();
        if (phase == PH_PAUSED) drawPause();          // 暂停时在游戏画面上盖一层遮罩
        if (phase == PH_GAMEOVER) drawGameOverScreen();  // 结束时盖结束遮罩
        break;
    }
}

// 绘制主菜单
static void drawMenu()
{
    TextCenter(fontBig, "俄罗斯方块", WIN_W / 2.0f, 110, 60, kGold);  // 大标题
    const char* items[3] = {"开始游戏", "排行榜", "退出游戏"};         // 三个菜单项
    for (int i = 0; i < 3; ++i)
    {
        // 计算第 i 个按钮的矩形（宽 260、高 56，水平居中，纵向间距 70）
        Rectangle b = Rectangle{(WIN_W - 260) / 2.0f, 260 + i * 70.0f, 260, 56};
        menuBounds[i] = b;        // 记录按钮区域，供鼠标悬停/点击检测
        bool on = i == menuSel;   // 是否为当前选中项
        if (on)
        {
            // 选中：画蓝色圆角底 + 一圈更亮的描边
            DrawRectangleRounded(b, 0.3f, 8, kSelect);
            DrawRectangleRoundedLinesEx(b, 0.3f, 8, 2.0f, Shade(kSelect, 1.4f));
        }
        // 选中项文字用白色，未选中用暗灰色
```

```
        TextCenter(fontUI, items[i], WIN_W / 2.0f, b.y + 12, 28, on ? WHITE :
kTextDim);
    }
    TextCenter(fontUI, "↑↓ 选择，回车确认，也可点击鼠标", WIN_W / 2.0f, 590, 18,
kTextDim);  // 底部操作提示
}

// 绘制难度选择界面
static void drawDifficulty()
{
    TextCenter(fontBig, "选择难度", WIN_W / 2.0f, 80, 46, kGold);  // 标题
    for (int i = 0; i < 3; ++i)
    {
        // 每个难度按钮：宽 380、高 78，纵向间距 100
        Rectangle b = Rectangle{(WIN_W - 380) / 2.0f, 200 + i * 100.0f, 380, 78};
        diffBounds[i] = b;       // 记录可点击区域
        bool on = i == diffSel;  // 是否被选中
        if (on)
        {
            DrawRectangleRounded(b, 0.18f, 8, kSelect);                        // 选中
蓝底
            DrawRectangleRoundedLinesEx(b, 0.18f, 8, 2.0f, Shade(kSelect, 1.4f));  //
亮描边
        }
        std::string name = std::string(kDiff[i].name) + "模式";  // 拼出"简单模式"等
文字
        TextCenter(fontUI, name.c_str(), WIN_W / 2.0f, b.y + 10, 26, on ? WHITE :
kText);  // 难度名
        TextCenter(fontUI, kDiff[i].desc, WIN_W / 2.0f, b.y + 44, 17, on ? kText :
kTextDim);  // 说明
    }
    TextCenter(fontUI, "↑↓ 选择，回车确认，Esc 返回", WIN_W / 2.0f, 600, 18, kTextDim);
// 底部提示
}

// 绘制排行榜
static void drawRank()
{
    TextCenter(fontBig, "排行榜", WIN_W / 2.0f, 65, 46, kGold);  // 标题
    for (int i = 0; i < RANK_SIZE; ++i)
    {
        float y = 155 + i * 80.0f;    // 每条记录的纵向位置，间距 80
        if (scores[i] > 0)
        {
```

```
            // 有成绩：拼出"第 X 名    YYY 分"
            std::string s = "第 " + std::to_string(i + 1) + " 名    " +
                            std::to_string(scores[i]) + " 分";
            TextCenter(fontUI, s.c_str(), WIN_W / 2.0f, y, 28, i == 0 ? kGold : kText);
// 第 1 名用金色
        }
        else
        {
            // 无成绩：显示"第 X 名    ---"，用暗灰色
            std::string s = "第 " + std::to_string(i + 1) + " 名    ---";
            TextCenter(fontUI, s.c_str(), WIN_W / 2.0f, y, 28, kTextDim);
        }
    }
    TextCenter(fontUI, "按任意键或点击鼠标返回", WIN_W / 2.0f, 600, 18, kTextDim);  //
底部返回提示
}

// 绘制游戏主画面：棋盘边框/网格、已锁定方块、影子、活动方块，最后画右侧信息栏
static void drawGame()
{
    // ---- 棋盘外框和内部背景 ----
    // 外框：比棋盘四周各大 4 像素的一圈灰紫色边框
    DrawRectangle(BOARD_X - 4, BOARD_Y - 4, BOARD_W * CELL + 8, BOARD_H * CELL + 8,
                  Color{42, 42, 56, 255});
    DrawRectangle(BOARD_X, BOARD_Y, BOARD_W * CELL, BOARD_H * CELL, kBoardBg);  //
棋盘内部底色
    for (int x = 1; x < BOARD_W; ++x)   // 画竖向网格线（每列一条，宽 1 像素）
        DrawRectangle(BOARD_X + x * CELL, BOARD_Y, 1, BOARD_H * CELL, kGrid);
    for (int y = 1; y < BOARD_H; ++y)   // 画横向网格线（每行一条）
        DrawRectangle(BOARD_X, BOARD_Y + y * CELL, BOARD_W * CELL, 1, kGrid);

    // ---- 已锁定在棋盘上的方块 ----
    for (int y = 0; y < BOARD_H; ++y)
    {
        for (int x = 0; x < BOARD_W; ++x)
        {
            if (cell[y][x] == -1) continue;        // 空格跳过
            Color c = kShapeColor[cell[y][x]];     // 按形状编号取颜色
            // 消行闪烁效果：正在被删除的行，每隔 0.07 秒在原色和白色之间切换
            if (phase == PH_CLEARING &&
                std::find(clearing.begin(), clearing.end(), y) != clearing.end())
            {
                if (std::fmod(clearTimer, 0.14f) < 0.07f) c = WHITE;  // 前半段变白，
后半段恢复
```

```
            }
            DrawCell(BOARD_X + x * CELL, BOARD_Y + y * CELL, CELL, c);  // 画这一格
        }
    }

    // ---- 活动方块和它的落点影子（仅在游戏中 / 暂停时显示）----
    if (phase == PH_PLAYING || phase == PH_PAUSED)
    {
        int sy = shadowY();    // 先算出影子落点的行
        // 先画影子（画在活动方块下层）
        for (int i = 0; i < 4; ++i)
            for (int j = 0; j < 4; ++j)
            {
                if (g_piece[shape][form][i][j] != 1) continue;
                DrawGhostCell(BOARD_X + (px + j) * CELL,
                              BOARD_Y + (sy + i) * CELL, CELL, kShapeColor[shape]);
            }
        // 再画活动方块本体
        for (int i = 0; i < 4; ++i)
            for (int j = 0; j < 4; ++j)
            {
                if (g_piece[shape][form][i][j] != 1) continue;
                int by = py + i;
                if (by < 0) continue;   // 还在屏幕外（出生区）的格子不画
                DrawCell(BOARD_X + (px + j) * CELL,
                         BOARD_Y + by * CELL, CELL, kShapeColor[shape]);
            }
    }

    drawPanel();   // 绘制右侧信息栏
}

// 绘制右侧信息栏：下一个方块预览、四项统计数据、操作说明、当前难度
static void drawPanel()
{
    float x = (float)PANEL_X;    // 信息栏左边界

    DrawTextEx(fontUI, "下一个", Vector2{x, 18}, 22, 1.0f, kText);  // 预览区标题

    // 预览框：加高到能竖排显示接下来的 3 个方块
    Rectangle pv = Rectangle{x + 5, 48, 190, 195};
    DrawRectangleRounded(pv, 0.08f, 8, Color{26, 26, 36, 255});       // 预览框深色底
    DrawRectangleRoundedLinesEx(pv, 0.08f, 8, 2.0f, Color{52, 52, 70, 255});  // 预览框描边
```

```
    drawNext(pv);    // 在框内画 3 个待出方块

    // 四个统计块（标签 + 数值），纵向每块间距 65
    statBlock(x, 260, "当前分数", score);
    statBlock(x, 325, "最高纪录", high);
    statBlock(x, 390, "当前等级", level);
    statBlock(x, 455, "消除行数", totalLines);

    // 操作说明（6 行，每行间距 20）
    const char* help[] = {"← → 移动", "↓ 加速", "↑/空格 旋转",
                          "P 暂停", "R 重开", "Esc 菜单"};
    for (int i = 0; i < 6; ++i)
        DrawTextEx(fontUI, help[i], Vector2{x, 505.0f + i * 20.0f}, 17, 1.0f,
kTextDim);

    std::string d = "难度：" + std::string(kDiff[diffIdx].name);  // 底部显示当前难度
    DrawTextEx(fontUI, d.c_str(), Vector2{x, 625}, 19, 1.0f, kGold);
}

// 画一个"标签 + 大数值"的统计小块（用于分数、最高分、等级、行数）
static void statBlock(float x, int y, const char* label, int value)
{
    DrawTextEx(fontUI, label, Vector2{x, (float)y}, 18, 1.0f, kTextDim);  // 上方：
暗灰色标签
    DrawTextEx(fontUI, std::to_string(value).c_str(),
               Vector2{x, (float)y + 24}, 26, 1.0f, kText);  // 下方：亮白色大数字
}

// 在预览框 pv 里竖排画队列最前面的 3 个方块（即接下来依次出现的 3 个）
static void drawNext(Rectangle pv)
{
    const int gs = 18;                    // 预览中每个小格的像素边长
    float slotH = pv.height / 3.0f;       // 预览框高度三等分，每个方块占一个槽位
    for (int k = 0; k < 3; ++k)
    {
        // 从队头开始，依次取第 k 个队列元素（环形下标要对容量取模）
        int idx = (qHead + k) % Q_CAP;
        int sh = pieceQueue[idx].shape;
        int fm = pieceQueue[idx].form;

        // 计算这个方块实际占用格子的包围盒（最小/最大行、列），
        // 目的是去掉 4×4 矩阵周围的空白，让预览方块紧凑居中
        int minR = 4, maxR = -1, minC = 4, maxC = -1;
        for (int i = 0; i < 4; ++i)
```

```
            for (int j = 0; j < 4; ++j)
                if (g_piece[sh][fm][i][j] == 1)
                {
                    minR = std::min(minR, i);   // 最上方有方块的行
                    maxR = std::max(maxR, i);   // 最下方有方块的行
                    minC = std::min(minC, j);   // 最左有方块的列
                    maxC = std::max(maxC, j);   // 最右有方块的列
                }
        float w = (maxC - minC + 1) * gs;   // 方块实际像素宽
        float h = (maxR - minR + 1) * gs;   // 方块实际像素高
        // 左上角坐标：槽位内水平居中、垂直居中
        float ox = pv.x + (pv.width - w) / 2;
        float oy = pv.y + k * slotH + (slotH - h) / 2;
        // 只在包围盒范围内逐格绘制
        for (int i = minR; i <= maxR; ++i)
            for (int j = minC; j <= maxC; ++j)
                if (g_piece[sh][fm][i][j] == 1)
                    DrawCell((int)(ox + (j - minC) * gs),
                             (int)(oy + (i - minR) * gs), gs, kShapeColor[sh]);
    }
}

// 绘制暂停遮罩：半透明黑层 + 居中提示文字
static void drawPause()
{
    DrawRectangle(0, 0, WIN_W, WIN_H, Color{0, 0, 0, 160});   // 全屏半透明黑色（alpha=160）
    TextCenter(fontBig, "游戏暂停", WIN_W / 2.0f, 250, 52, kText);
    TextCenter(fontUI, "按 P 继续", WIN_W / 2.0f, 330, 22, kText);
    TextCenter(fontUI, "Esc 返回主菜单", WIN_W / 2.0f, 375, 20, kTextDim);
}

// 绘制游戏结束遮罩
static void drawGameOverScreen()
{
    DrawRectangle(0, 0, WIN_W, WIN_H, Color{0, 0, 0, 170}); // 半透明黑色遮罩
    TextCenter(fontBig, "游戏结束", WIN_W / 2.0f, 170, 52, Color{235, 95, 95, 255});
// 红色标题
    std::string s = "本局得分：" + std::to_string(score); // 拼出本局得分
    TextCenter(fontUI, s.c_str(), WIN_W / 2.0f, 260, 28, kText);
    TextCenter(fontUI, "成绩已记入排行榜", WIN_W / 2.0f, 310, 19, kTextDim);
    TextCenter(fontUI, "R 重新开始", WIN_W / 2.0f, 400, 22, kText);
    TextCenter(fontUI, "Esc 返回主菜单", WIN_W / 2.0f, 445, 20, kTextDim);
}
```

```
// ============================================================
// 十二、主程序入口
// ============================================================
int main()
{
    // ---- 初始化窗口与画面 ----
    InitWindow(WIN_W, WIN_H, "俄罗斯方块");  // 创建 800×650 的窗口，标题"俄罗斯方块"
    SetTargetFPS(60);                        // 目标帧率 60 FPS（游戏运行流畅度）

    InitPieces();                            // 初始化 7 种形状的全部旋转矩阵

    // ---- 加载中文字体 ----
    // 注意：Raylib 对微软雅黑 msyh.ttc（TTC 集合）支持有问题，这里改用黑体 simhei.ttf
    std::vector<int> cp = CollectCodePoints();                        // 收集界面所需全部码点
    const char* fontPath = "C:\\Windows\\Fonts\\simhei.ttf";          // Windows 自带黑体路径
    fontUI = LoadFontEx(fontPath, 26, cp.data(), (int)cp.size());    // 加载 26 号普通字体
    fontBig = LoadFontEx(fontPath, 56, cp.data(), (int)cp.size());   // 加载 56 号大字体
    if (fontUI.texture.id == 0 || fontBig.texture.id == 0)
    {
        // 字体加载失败（纹理 id 为 0）：退回 raylib 默认字体，避免程序崩溃
        fontUI = GetFontDefault();
        fontBig = GetFontDefault();
    }

    // ---- 读取最高分和排行榜存档 ----
    std::ifstream in(SCORE_FILE);    // 打开最高分文件
    int h = 0;
    if (in >> h) high = std::max(0, h);  // 读到数值就作为最高分（负数按 0 处理）
    loadRanks();                         // 读取排行榜

    // ---- 主循环（每帧执行一次）----
    // WindowShouldClose：点窗口关闭按钮时为 true；wantExit：菜单选了退出
    while (!WindowShouldClose() && !wantExit)
    {
        float dt = GetFrameTime();   // 获取本帧时间增量（秒）
        update(dt);                 // 1. 更新逻辑（处理输入、移动、计时、状态切换）
        BeginDrawing();              // 2. 开始绘制（raylib 要求绘图包在 Begin/End 之间）
        draw();                     // 3. 绘制本帧画面
        EndDrawing();               // 4. 结束绘制并把画面刷新到屏幕
```

```
    }

    // ---- 退出前清理资源 ----
    saveHigh();             // 最后再保存一次最高分
    UnloadFont(fontUI);     // 卸载普通字体（释放显存）
    UnloadFont(fontBig);    // 卸载大字体
    CloseWindow();          // 关闭窗口、销毁绘图上下文
    return 0;               // 程序正常结束
}
```